

Monitoreo de infraestructura con Zabbix

Una plataforma dockerizada con Zabbix 6.x, alertas, backend real, métricas, SLO y pruebas de carga en vivo.

4

hosts monitoreados

7

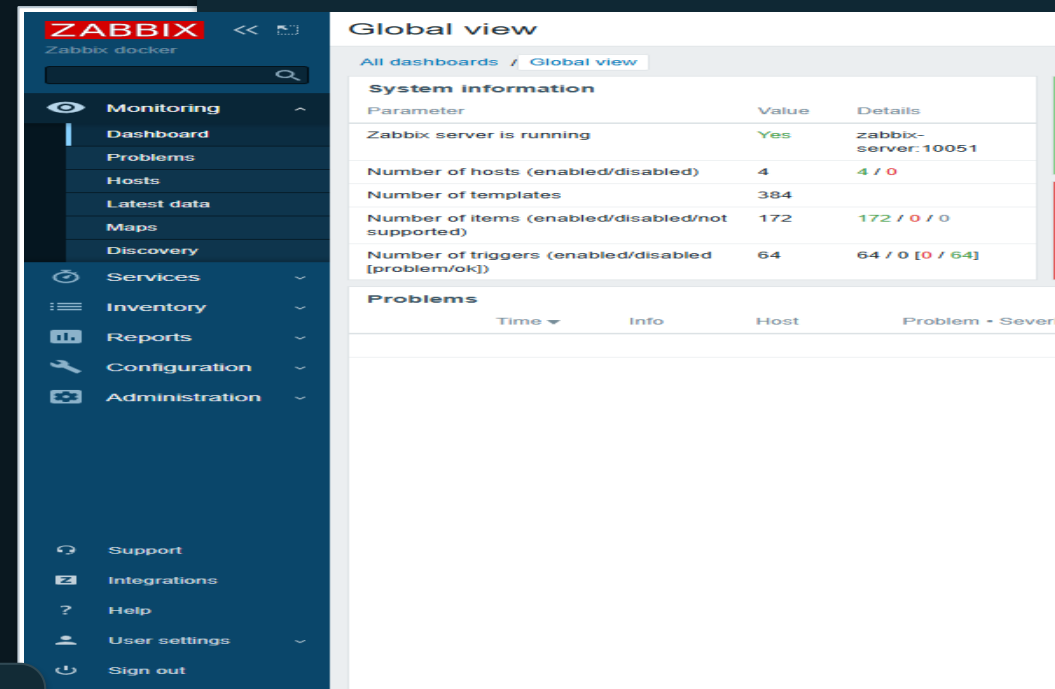
páginas IEEE

96

requests smoke

0

fallas auditoría



Juan Camilo Ballesteros Sierra
Luis Felipe Murillo Matallana
Juan Sebastián Delgado
Daniela Castro Quiñones

Sin observabilidad, la falla se detecta tarde y sin evidencia.

La infraestructura evaluada mezcla web, base de datos, DNS y FTP. Si un componente cae, el usuario percibe degradación antes de que el equipo tenga una causa clara.

Riesgo operativo

No basta saber que el servidor está encendido; se debe medir servicio, recursos, alertas e histórico.

Falla

servicio cae

Usuario

reporta tarde

Operación

revisa manual

Histórico

no existe

Criterio de evaluación

La rúbrica pide diseño, proceso de validación y construcción funcional. Por eso la demo debe probar el flujo completo.

La entrega es una plataforma observable, no una maqueta.

Mínimo del enunciado

Zabbix Server, base de datos, frontend, MailHog, cuatro hosts, agentes, templates, triggers y dashboards.

Valor agregado

Portal HTTPS con backend Node.js, MariaDB, gráficas, SLO, exporter /metrics, Artillery y auditoría reproducible.

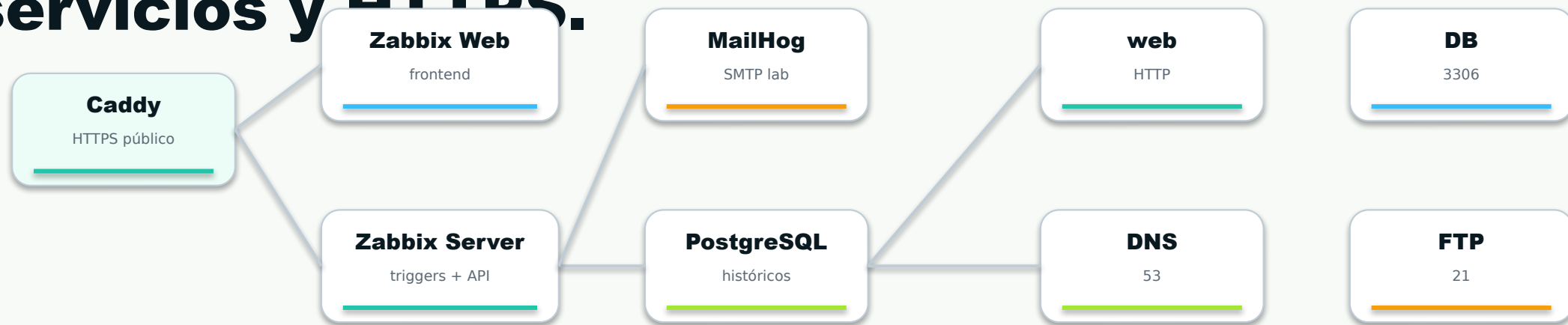
Evidencia defendible

Repositorio, README, informe IEEE de 7 páginas, PPTX, capturas, scripts y matriz /api/compliance.

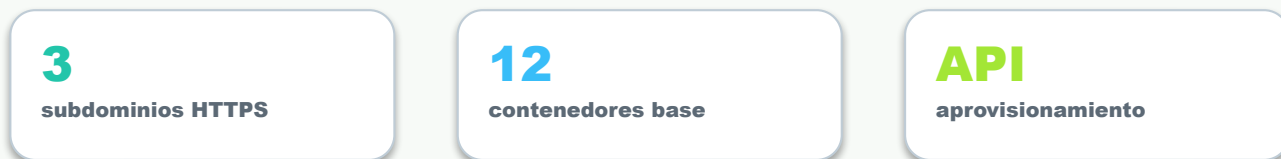
Tesis de la sustentación

Zabbix observa una infraestructura real, detecta fallas y permite explicar el comportamiento del sistema bajo carga.

Compose conecta monitoreo, servicios y HTTPS.



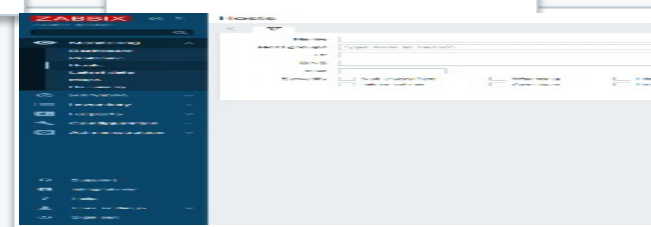
Infraestructura monitoreada



Cuatro servicios distintos quedan medidos como hosts operativos.

Host	Servicio	Check	Agente
web-host	Portal Node.js	HTTP puerto 80	web-agent
db-host	MariaDB	TCP puerto 3306	db-agent
dns-host	CoreDNS	TCP puerto 53	dns-agent
ftp-host	VSFTPD	FTP puerto 21	ftp-agent

El inventario se puede defender desde Zabbix y desde Compose: cada host tiene servicio, puerto, agente y trigger asociado.



El despliegue se puede reconstruir desde archivos, no desde pasos

n

docker-compose.yml

topología local

docker-compose.vps.yml

publicación VPS

Dockerfile Zabbix

imagen personalizada

provision_zabbix.py

hosts, items y triggers

audit-project.sh

validación reproducible

Compose

orquesta

API

aprovisiona

Zabbix

observa

MailHog

notifica

Punto fuerte

La solución queda trazable: archivo -> contenedor -> objeto
Zabbix -> evidencia -> auditoría.

El dashboard resume la operación.

ZABBIX Zabbix docker

Global view

All dashboards / Global view

Parameter	Value	Details
Zabbix server is running	Yes	zabbix-server:10051
Number of hosts (enabled/disabled)	4	4 / 0
Number of templates	384	
Number of items (enabled/disabled/not supported)	172	172 / 0 / 0
Number of triggers (enabled/disabled [problem/ok])	64	64 / 0 [0 / 64]

Problems

Time	Info	Host	Problem	Severity
------	------	------	---------	----------

CPU

recursos

RAM

memoria

DISCO

capacidad

HTTP

servicios

Latest data prueba que Zabbix recibe señales de hosts y servicios.

ZABBIX Zabbix docker

Monitoring

- Dashboard
- Problems
- Hosts
- Latest data**
- Maps
- Discovery

Services

- Inventory
- Reports
- Configuration
- Administration

Support

- Integrations
- Help
- User settings
- Sign out

Latest data

Host groups: type here to search

Hosts: type here to search

Name:

Subfilter affects only filtered data

HOSTS

[db-host](#) 43 [dns-host](#) 43 [ftp-host](#) 43 [web-host](#) 43

TAGS

[component](#) 168

TAG VALUES

[component](#): [application](#) 4 [cpu](#) 68 [environment](#) 4 [memory](#) 28 [os](#) 12 [sec](#)

DATA

[With data](#) [Without data](#)

☐ Host	Name ▲
☐ dns-host	Disponibilidad DNS dns-service TCP
☐ ftp-host	Disponibilidad FTP ftp-service
☐ web-host	Disponibilidad HTTP web-service
☐ db-host	Disponibilidad MySQL db-service
☐ web-host	Linux: Available memory ?
☐ db-host	Linux: Available memory ?
☐ dns-host	Linux: Available memory ?
☐ ftp-host	Linux: Available memory ?
☐ web-host	Linux: Available memory in % ?

Checks básicos

agent.ping, CPU, memoria y disco para observar salud del host.

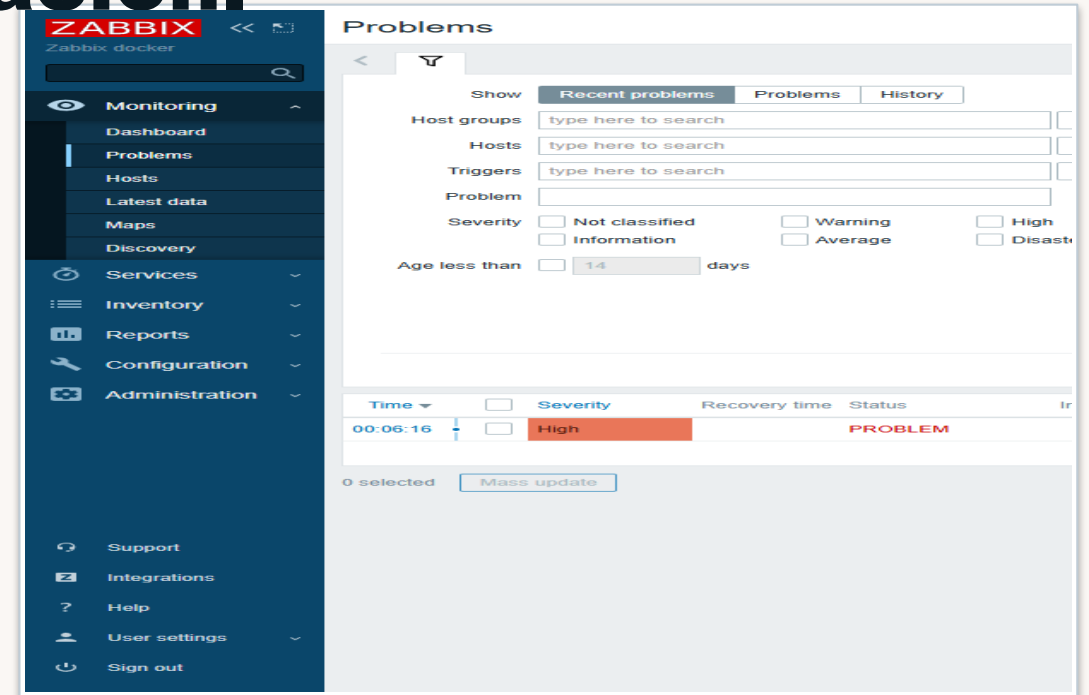
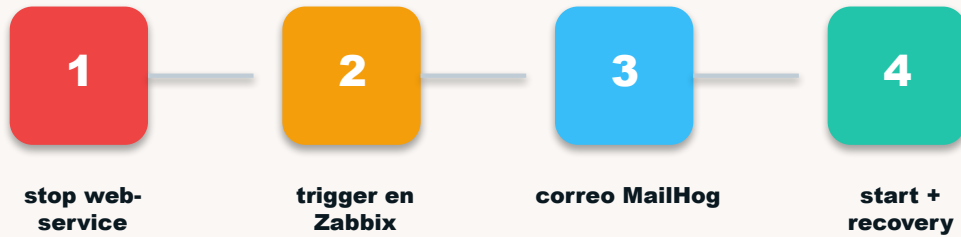
Checks de servicio

HTTP, MySQL/MariaDB, DNS y FTP validan disponibilidad funcional.

Checks avanzados

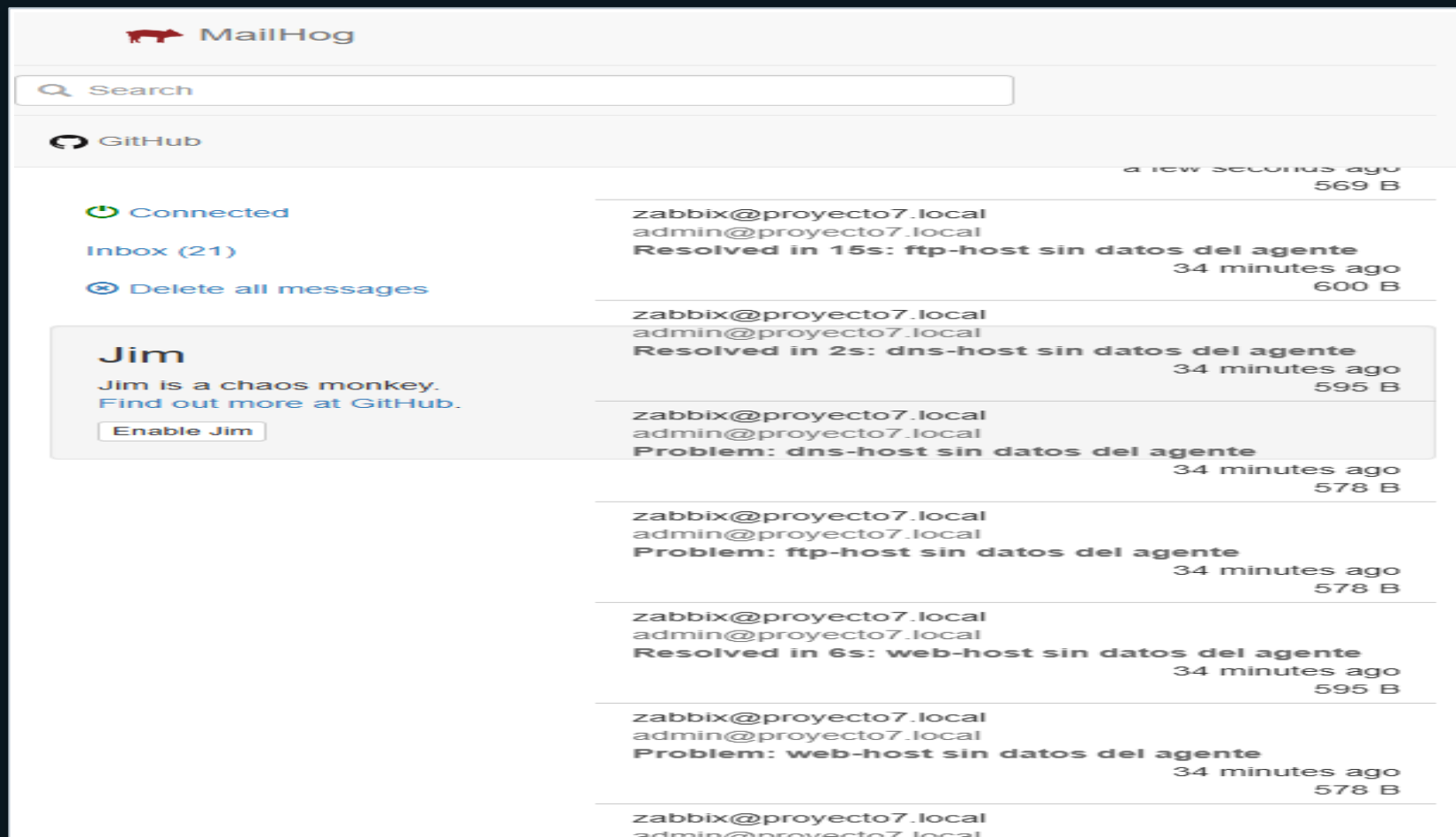
/metrics, /api/db/status y web scenario público conectan Zabbix con la app real.

La prueba clave recorre falla, problema, alerta y recuperación.



El valor de esta prueba está en el flujo completo: no solo aparece el problema, también queda histórico y evidencia de recuperación.

MailHog evidencia las alertas.



Laboratorio seguro

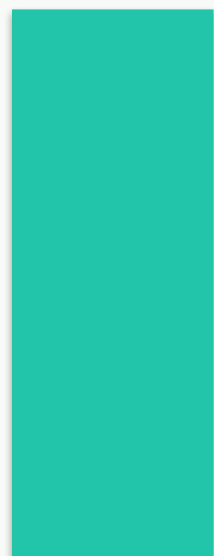
Captura correos sin enviar spam a cuentas reales durante las pruebas.

Escalamiento

Queda documentado un canal SMTP real del dominio para producción.

Artillery convierte la demo en una prueba de carga.

artillery run tests/artillery-live-demo.yml



96

requests smoke



0

usuarios fallidos



24

p95 aprox. ms

Qué demuestra

El backend registra telemetría e incidentes en MariaDB mientras Zabbix observa endpoints públicos, DB status y /metrics.

Cómo se ve en vivo

Durante la sustentación suben requests, rutas golpeadas, cargas recientes y SLO en el portal.

La evaluación queda cubierta con evidencia y verificación reproducible

100%

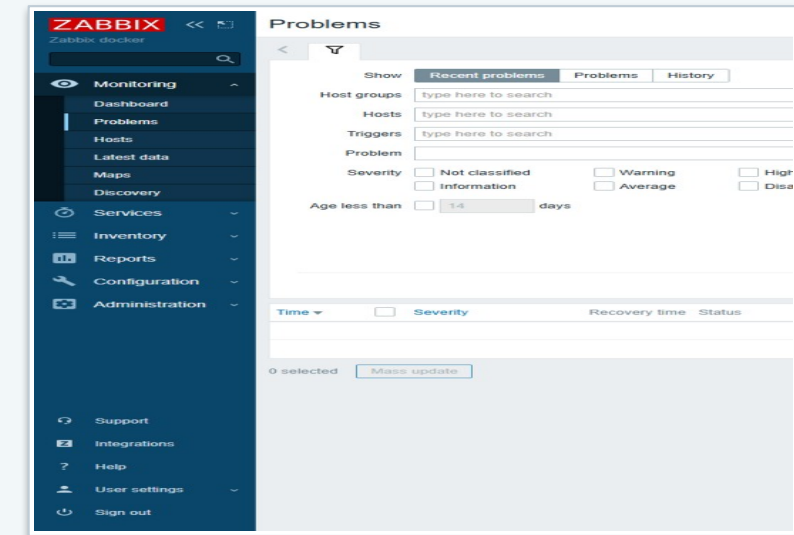
/api/compliance

27

validaciones OK

0

fallas auditoría

 Dashboard en tiempo real Simulación de caída Alertas en MailHog Métricas históricas Carga Artillery Informe IEEE + repo

El proyecto se entrega como repositorio, informe, deck y demo pública

Informe IEEE

entrega-final/Informe_IEEE_Proyecto7_Zabbix.pdf

Diapositivas

entrega-final/Presentacion_Proyecto7_Zabbix.pptx

Repositorio

github.com/ballesterossmartsolutionssas/Proyecto7-Zabbix

Evidencias

entrega-final/evidencias + auditoría

Reparto: Luis Felipe abre, Juan Sebastián explica arquitectura, Daniela muestra Zabbix y Juan Camilo cierra con pruebas en vivo.

La demo cierra con tráfico, caída, alerta y auditoría.

1

Abrir portal`https://web-zabbix.negociocontigo.com`

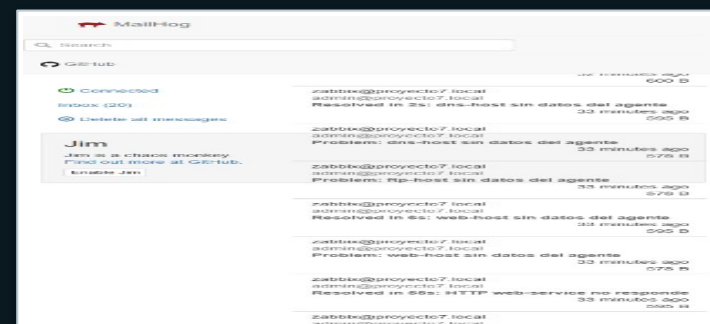
2

Generar carga`artillery run tests/artillery-live-demo.yml`

3

Simular caída`docker compose -f docker-compose.vps.yml stop web-service`

4

Validar cierre`bash scripts/audit-project.sh`

Cierre: Zabbix detecta, MailHog evidencia, Artillery presiona y la auditoría confirma cumplimiento.